

GMX · SECURITY REVIEW

GMX

Synthetics Updates.

A security review of Synthetics Updates.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
GMX	Synthetics Updates	Guardian	October 14 – October 25, 2025	November 3, 2025

§ 00

Contents.

38 pages — overview, confidence ranking, scope, 25 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 13 findings	P. 09
	Remediation Review · 12 findings	P. 24
§ 05	Disclaimer	P. 37
§ 06	About Guardian	P. 38

§ 01

Overview.

Guardian reviewed Synthetics Updates across 2 review rounds. The final report includes 2 High, 4 Medium, 17 Low, 2 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	gmx-io/gmx-synthetics
CLIENT	METHODOLOGY
GMX	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Synthetics Updates	6e706f2a3fb5
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	f04992d03cbf
AUDIT WINDOW	
October 14 – October 25, 2025	
FINAL REPORT	
November 3, 2025	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	0	0	0	0	0
■ High	2	0	0	0	2
■ Medium	4	0	4	0	0
■ Low	17	0	14	0	3
■ Informational	2	0	2	0	0
All severities	25	0	20	0	5

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Synthetics Updates in `gmx-io/gmx-synthetics`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Virtual Inventory Does Not Track Correctly	Logical Error	HIGH	RESOLVED	p. 9
H-02	Funding Is Broken Due To Asymmetry	Logical Error	HIGH	RESOLVED	p. 10
M-01	Longs Incorrectly Share Borrowing Fees	Logical Error	MEDIUM	ACKNOWLEDGED	p. 12
M-02	Position Health Changed On Upgrade	Warning	MEDIUM	ACKNOWLEDGED	p. 13
M-03	Funding Fees Are Incorrectly Distributed	Logical Error	MEDIUM	ACKNOWLEDGED	p. 14
L-01	Liquidation Preference Hijacked	Unexpected Behavior	LOW	RESOLVED	p. 15
L-02	Max Price Is Always Used	Unexpected Behavior	LOW	RESOLVED	p. 16
L-03	Position Health Can Decrease On Collateral Deposits	Unexpected Behavior	LOW	RESOLVED	p. 17
L-04	Stepwise Pool Value Changes	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 18
L-05	Likely Impact Pool Dearth	Warning	LOW	ACKNOWLEDGED	p. 19
L-06	Funding Is Charged Inaccurately Retroactively	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 20
L-07	Saved Funding Factor Rate Reset	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 21
I-01	Funding And Borrowing Fees Can Prevent Deposit	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDATION REVIEW					
M-01	Positions May Be Prevented From Closing	DoS	MEDIUM	ACKNOWLEDGED	p. 24
L-01	Lacking OI Notional Caps	Validation	LOW	ACKNOWLEDGED	p. 25
L-02	Virtual Inventory Migration Risks	Warning	LOW	ACKNOWLEDGED	p. 26
L-03	One Way Migration Process Warning	Warning	LOW	ACKNOWLEDGED	p. 28
L-04	oracleProviderUpdatedAt Ineffective Validation	Validation	LOW	ACKNOWLEDGED	p. 29
L-05	Virtual Inventory Keys Are Allowed Base Keys	Warning	LOW	ACKNOWLEDGED	p. 30
L-06	Dangerous OI In Tokens Base Key	Warning	LOW	ACKNOWLEDGED	p. 31
L-07	Positive Price Impact Capped On Cost	Warning	LOW	ACKNOWLEDGED	p. 32
L-08	Negative Price Impact Capped On Cost	Warning	LOW	ACKNOWLEDGED	p. 33
L-09	Price Impact Pool Gaming Risk	Gaming	LOW	ACKNOWLEDGED	p. 34
L-10	MaxCollateralSum Decrease DoS	DoS	LOW	ACKNOWLEDGED	p. 35
I-01	Decrease Flow Always Uses Initial Margin Factor	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 36

HIGH **H-01**

RESOLVED

Virtual Inventory Does Not Track Correctly

CATEGORY	Logical Error
LOCATION	PositionPricingUtils.sol
REVIEW	Main Review

DESCRIPTION

The Virtual Inventory is still maintained based upon USD values in the `MarketUtils.applyDeltaToOpenInterest` function instead of being based on size in token values and factoring in the current token price.

However in the `getPriceImpactUsd` function the `params.usdDelta` in memory is updated to reflect the size in token delta valued at the corresponding index token price when the `USE_OPEN_INTEREST_IN_TOKENS_FOR_BALANCE` value is true.

This will result in reverts and inaccurate price impact calculations throughout the exchange.

RECOMMENDATION

The Virtual Inventory must now be tracked in token sizes and factor the current index price in it's logic. This poses a sensitive migration from the existing values stored for live virtual inventories.

RESOLUTION

Status: Resolved.

HIGH **H-02**

RESOLVED

Funding Is Broken Due To Asymmetry

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

DESCRIPTION

The update of funding relying on the current notional value of open interest now requires a mirroring update to the distribution units of funding fees. Currently the funding fee amount charged is not in agreement with the funding that is computed by the funding algorithm due to asymmetry in the way that the per unit funding is distributed and the way it is collected.

- Index token price is \$2
- \$200 OI Cost Long, token size 100
- \$100 OI Cost Short, token size 50
- Index token price goes to \$1.50
- \$150 OI Notional Long
- \$75 OI Notional Short
- On the long side there is a position using the long token as collateral with 65 size in tokens and \$125 in cost, and a position using the short token as collateral with 35 size in tokens and \$75 in cost
- On the short side there is one position using the short token as collateral with 50 size in tokens and \$100 in cost
- Funding determines that \$10 is to be paid from longs to shorts
- $\text{fundingUsdForLongCollateral} = \$10 * 65 / 100 = \$6.5$
- $\text{fundingUsdForShortCollateral} = \$10 * 35 / 100 = \$3.5$
- $\text{result.fundingFeeAmountPerSizeDelta.long.longToken} = \$6.5 / 97.5 \text{ OI Notional} = 0.0667 \text{ per Notional OI}$
- $\text{result.fundingFeeAmountPerSizeDelta.long.shortToken} = \$3.5 / 52.5 = \$0.07 \text{ per Notional OI}$
- $\text{result.claimableFundingAmountPerSizeDelta.short.longToken} = \$6.5 / \$75 = 0.0867 \text{ per Notional OI}$
- $\text{result.claimableFundingAmountPerSizeDelta.short.shortToken} = \$3.5 / \$75 = 0.0467 \text{ per Notional OI}$
- However these claimable factors are not partitioned out based upon notional OI, but instead the cost OI, therefore the contracts will end up paying out more than the \$10 in

H-02 Funding Is Broken Due To Asymmetry

CONTINUED

funding:

- `result.claimableFundingAmountPerSizeDelta.short.longToken` is applied per cost OI for all shorts: $0.0867 * \$100 = \8.67
- `result.claimableFundingAmountPerSizeDelta.short.shortToken` is applied per cost OI for all shorts: $0.0467 * \$100 = \4.67
- The summation of total funding received by the shorts is $8.67 + 4.67 = 13.34$ which is significantly larger than the amount of funding computed
- Notice that in this calculation we have ignored the conversion back to collateral tokens for the funding per size for the sake of example
- If the funding were to be partitioned out based upon the current notional size:
 - `result.claimableFundingAmountPerSizeDelta.short.longToken` is applied per cost OI for all shorts: $0.0867 * \$75 = \6.50
 - `result.claimableFundingAmountPerSizeDelta.short.shortToken` is applied per cost OI for all shorts: $0.0467 * \$75 = \3.50
 - The aggregate funding received is the expected $\$6.50 + \$3.50 = \$10$
 - However funding cannot be paid out this way because notional sizes change over time based on index token price, instead the funding per size should be stored in per size units of "per index sizeInToken".

— RECOMMENDATION

Refactor the funding per size logic on the distribution side and the collection side throughout GMX to use a `perSizeInToken` over a `perSizeInCost` model.

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

ACKNOWLEDGED

Longs Incorrectly Share Borrowing Fees

CATEGORY Logical Error
LOCATION MarketUtils.sol
REVIEW Main Review

DESCRIPTION

The borrowing fees are attributed to positions based on their open interest contributions in position "cost" rather than their current position notional value.

This can be seen in the `MarketUtils.getBorrowingFees` function where the position `sizeInUsd` with the difference in the `cumulativeBorrowingFactor` produces the resulting borrowing fee USD amount.

However, the `sizeInUsd` is not exactly what contributes to the borrowing fees charged to a side, instead it is the `reservedUsd`.

For shorts, this `reservedUsd` is the cost basis of the positions, so the borrowing fees being applied based on the `sizeInUsd` is correct.

However for longs, the `reservedUsd` is the notional value of the long open interest in tokens. As a result, the borrowing fees are not necessarily spread out amongst long positions fairly. There may be a long position that entered while prices were low and has a low "cost" but a large current notional value, and is thus reserving a larger share of the pool backing value than a position that has a higher cost but a lower notional value.

The correct distribution of borrowing fees for longs would be to charge the position with the higher notional value a higher share of the borrowing fees. But in this example that position is actually charged a lower share of the borrowing fees.

RECOMMENDATION

Changing this logic may be too risky and garner too much complexity at this point. However be aware of this inexact distribution of borrowing fees on the exchange.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Position Health Changed On Upgrade

CATEGORY Warning
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The upgrade will change the impact values that are computed in the `isPositionLiquidatable` function as well as the impact that would be realized upon decreasing or liquidating the position.

This means the effective health of positions are changed immediately by this upgrade, and it may be the case that non-liquidatable positions could be made liquidatable or even insolvent. And currently liquidatable positions could be made healthy.

This may pose a risk to the health of markets on the exchange and if not carefully orchestrated, could cause bad debt in a GM market.

— RECOMMENDATION

Ensure that there are no positions that would experience a large change in the price impact that they would receive and thus do not have a significant change in health.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Funding Fees Are Incorrectly Distributed

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

DESCRIPTION

Now that funding is determined based upon the current notional value of each side, those positions which contribute more towards the current notional value should be charged and awarded accordingly.

Currently, the charging and awarding of funding fees is based on the `sizeInUse` of the position. However consider the following scenario:

- Longs have \$100 total notional size
- Shorts have \$50 total notional size
- Longs pay shorts \$10 of funding
- Bob is a long with \$60 of cost and \$40 of notional size
- Alice is a long with \$40 of cost and \$60 of notional size
- Bob pays \$6 of funding while Alice pays \$4 of funding, even though Alice contributes 50% more to the current notional size

RECOMMENDATION

Consider if the funding distribution should be updated to be based upon the notional size of positions instead of the cost.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Liquidation Preference Hijacked

CATEGORY Unexpected Behavior
LOCATION Global
REVIEW Main Review

— DESCRIPTION

When a position is insolvent there is a particular order in which debts are prioritized in the `DecreasePositionCollateralUtils` file, however now because position increase orders with only collateral can be executed even when the position is insolvent, this liquidation preference can be bypassed.

This is because the funding and borrowing fees are charged to the user's position collateral in the `processCollateral` function on increase.

The funding fees carry the highest priority on liquidation, however the borrowing fees carry a lower priority than paying the PnL. This may be unexpected as a portion of the remaining collateral from the position can now go towards the borrowing fee receiver instead of to paying down the outstanding PnL.

— RECOMMENDATION

Be aware of this bypass of the liquidation preference for insolvent positions, and consider adding back a portion of the validation in the `validatePosition` function, such as the minimum collateral USD check to ensure that insolvent positions cannot be updated to bypass this liquidation preference.

— RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Max Price Is Always Used

CATEGORY Unexpected Behavior
LOCATION Global
REVIEW Main Review

DESCRIPTION

When computing the `swapPriceImpact`, the USD value of each side of the pool is computed using the `midPrice` of the provided price.

However for the position price impact as well as the funding and borrowing sum position notional calculations, the max price of the index token is used. When purely comparing sides to discern which is larger, this has no effect since both are using the max price.

But for calculations such as price impact or funding, this can create a difference as the delta in tokens is worth more in USD if the maximum price is used. If there is a spread being used for the relevant index token in the action, then this spread will have an impact on the resulting funding and impact that is charged.

RECOMMENDATION

Consider if the `midPrice` should be used, similar to how it is used in the `swapPriceImpact` calculations to limit the effect of the spread on these operations.

RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

Position Health Can Decrease On Collateral Deposits

CATEGORY	Unexpected Behavior
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

When computing the borrowing fee amount that will be charged in collateral units to the user's collateral, the borrowing fee amount in USD is divided by the price that is stored in the Oracle contract for the execution of the order.

This price effectively determines how much a user pays for borrowing fees, and therefore if an order is executed with prices from 5 minutes ago, then it could result in the account paying more or less borrowing fees (effective) than if that order were to be executed with the current prices, or if a liquidation were to occur at the current prices.

In some cases this may allow a position to decrease their health, as measured by current prices, with the execution of an increase order that only adds a small amount of collateral and is being executed with slightly stale prices.

— RECOMMENDATION

This is fine when a position is safely healthy, however it would be best to explicitly prevent a position from becoming insolvent even after an order is executed that only adds collateral, because of this edge case. Consider keeping all of the validations that occur in `validatePosition` for all increase orders and only optionally performing the leverage validation based on if the order has a nonzero `sizeDelta` or not.

— RESOLUTION

Status: Resolved.

LOW **L-04**

ACKNOWLEDGED

Stepwise Pool Value Changes

CATEGORY	Unexpected Behavior
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The pool value may change in a stepwise fashion during this upgrade since the definition of the smaller side for borrowing fees has changed.

Consider the following scenario:

- Skip smaller side is true
- Longs have 10 pending borrowing fees in aggregate, if they were to be charged
- Shorts have 5 pending borrowing fees in aggregate, if they were to be charged
- Longs have a total "cost" of 100 USD, and a notional of \$80
- Shorts have a total "cost" of 90 USD, and a notional of \$110
- Before this upgrade takes place, the smaller side is shorts, and the pending fees are 5
- After this upgrade takes place, the smaller side is longs, and the pending fees are 10

This creates an immediate change in the value of the pool and thus the price of the GM tokens in the single transaction that enables the new token open interest valuation.

This may create issues for integrators who accept GM tokens as collateral, due to a stepwise reduction in value, or small arbitrage opportunities due to a stepwise increase in value.

— RECOMMENDATION

Be aware of this risk when performing the upgrade.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Likely Impact Pool Dearth

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

In the past, it has been an invariant that the position impact pool does not pay out more than it receives paid in. With the new impact system there are few exceptions to this that are accepted, however still generally this ideal holds.

This upgrade threatens this status for live GM markets, since the pending impact that has been already computed and stored for open positions was computed using the previous impact calculations.

The new impact calculations could allow a net positive impact to be paid out upon all of the open positions when decreasing. This would leave a dearth of a lent amount on the market which does not get repaid even after all positions have closed.

— RECOMMENDATION

Be aware of this risk of introducing a dearth lent amount, and be prepared to use the admin functions to reduce lent amount as needed.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Funding Is Charged Inaccurately Retroactively

CATEGORY Unexpected Behavior
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Because the funding fees are now based upon the current notional value of each side, which factors the current market price. Whatever the price is at the time of the action that is updating the funding is, determines what the funding is that was charged over the entire period after the last funding update.

Consider the following scenario:

- At time 100 there is a funding update, price is \$100
- At time 110 the price is still \$100
- At time 111 there is a second funding update, price is \$105

In this example, the funding is charged as if the price has been \$105 for the entire period from [100, 110] .

— RECOMMENDATION

Be aware of this behavior, it may be fine as long as actions are frequent.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Saved Funding Factor Rate Reset

CATEGORY Unexpected Behavior
LOCATION MarketUtils.sol: 1314
REVIEW Main Review

DESCRIPTION

The `updateFundingState` function relies on the `getNextFundingAmountPerSize` function to return a result object which contains the `nextSavedFundingFactorPerSecond` that will be written to storage.

In the case where either long or short open interest is zero, the `getNextFundingAmountPerSize` function returns an empty result object which uses a default uint value of 0 for the `nextSavedFundingFactorPerSecond`.

As a result, the saved funding factor is always reset to zero, no matter what its value was before the funding update.

This may allow for a minor gaming of the funding rate in an edge case where there is only one large position that is paying funding fees on the paying side. Assuming the adaptive funding rate model is enabled, and that it slowly ratchets up the funding rate over time. The single large position on the paying side may strategically close their position and re-open it so that there is a funding update where the paying side goes to zero open interest and triggers this early return in the funding flow, resetting the saved funding factor per second value.

The position would then be re-opened and the funding increase would start to occur at a slower rate until the funding factor per second was ratcheted back up to the previous value. Compared to the base scenario without closing and re-opening the position, this position on the larger OI side would pay less funding fees overall, but likely more position order related fees.

RECOMMENDATION

Consider returning the current saved funding factor per second in the early return case in the `getNextFundingAmountPerSize` function.

L-07 Saved Funding Factor Rate Reset

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Funding And Borrowing Fees Can Prevent Deposit

CATEGORY Warning
LOCATION IncreasePositionUtils.sol
REVIEW Main Review

— DESCRIPTION

During the increase flow in the `processCollateral` function the outstanding fees are deducted from the `collateralDelta`.

As a result, it is possible for the `collateralDelta` to be negative, and in some cases where positions are in a profit to be more negative than the existing collateral for the position. These cases will result in a revert, disallowing the order even if it adds only collateral and not size to the position.

— RECOMMENDATION

Simply be aware that these cases can arise which prevent collateral addition for positions with high outstanding fees that are supported by profit.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Positions May Be Prevented From Closing

CATEGORY DoS
LOCATION PositionPricingUtils.sol: 348
REVIEW Remediation Review

DESCRIPTION

During the price impact computation flow the `getNextOpenInterestParams` function will revert if the provided `params.usdDelta` value is greater than the long or short open interest.

There is an edge case where this can be possible which prevents the last position holder from entirely closing their position.

When a market has homogenous backing tokens the `getOpenInterest` result is divided by a divisor of 2, which can lead to rounding error on the order of 1 wei. Since rounding occurs in the downward direction, the resulting open interest can understate the actual open interest in the market by 1 wei.

Consider the following example:

- Trader A has the only long position in a market with a `sizeInUsd` of $10e18+1$ and attempts to close it all
- The result of `getOpenInterest` reports: $(10e18+1)/2 + (10e18+1)/2 = 10e18$
- The `getNextOpenInterestParams` confirms that $(-params.usdDelta).toUint256() > longOpenInterest \Rightarrow 10e18+1 > 10e18$
- Results in a revert `UsdDeltaExceedsLongOpenInterest`

RECOMMENDATION

If the `usdDelta` is 1 wei larger than the long or short open interest in the `getNextOpenInterestParams` function, consider assigning the `nextLongOpenInterest` or `nextShortOpenInterest` to 0 to avoid reverts in this case.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Lacking OI Notional Caps

CATEGORY	Validation
LOCATION	Global
REVIEW	Remediation Review

DESCRIPTION

In the `validateOpenInterest` function that caps open interest within the `applyDeltaToOpenInterest` function the open interest is validated based on cost basis.

However in the `applyDeltaToOpenInterestInTokens` function the open interest is not validated based on the tokens exposure, nor the current notional value.

RECOMMENDATION

Consider if there should be validation caps on the open interest in the `applyDeltaToOpenInterestInTokens` to limit open interest in two ways:

- Raw token amount open interest caps
- Current value notional open interest caps

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Virtual Inventory Migration Risks

CATEGORY	Warning
LOCATION	—
REVIEW	Remediation Review

— DESCRIPTION

The virtual inventory tracking for the size in tokens with the `getVirtualInventoryForPositionsInTokens` function relies on the `virtualInventoryForPositionsInTokensKey` key which is not synced with the current token exposures of a market, therefore necessitating a manual syncing process.

If the virtual inventory is not synced properly it would perturb the incentive to balance the index token market group sufficiently, and may end up punishing traders more than they ought to be when receiving negative impact. Furthermore, it can also result in unjust liquidations, as the price impact result is included in the formula for an account's health.

The migration process is therefore very sensitive and must be carried out at the same time in which the tokens open interest usage is turned on.

If at any point during the migration process a position is closed, opened, increased, or decreased, changing the `openInterestInTokens` for a market before the `USE_OPEN_INTEREST_IN_TOKENS_FOR_BALANCE` key is enabled, the values that seeded the virtual inventory in tokens will be incorrect.

— RECOMMENDATION

Ideally the migration would use a single atomic transaction onchain which computes the correct net market exposure for the virtual inventory for every group and updates the virtual inventory for tokens key storage accordingly all in the same single transaction as the `USE_OPEN_INTEREST_IN_TOKENS_FOR_BALANCE` key is enabled.

This way there is no opportunity for a position change to perturb the new virtual inventory in tokens values. Alternatively, consider pausing trading activities for a brief period while the migration is occurring.

L-02 Virtual Inventory Migration Risks

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

One Way Migration Process Warning

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review

DESCRIPTION

In the `applyDeltaToOpenInterest` function an if conditional has been added to only invoke the `applyDeltaToVirtualInventoryForPositions` when `useOpenInterestInTokens` is false.

This creates a one-way door with respect to the migration process because if the virtual inventory based on the cost-basis is not being updated in lock-step with every position update that occurs, it will become out of sync.

This means once the `useOpenInterestInTokens` value is set to true, it can never be back-stepped to false because the old cost-basis virtual inventory will be out of sync and will not correctly apply price impact for trades, and may even cause liquidations to occur when they rightfully shouldn't.

RECOMMENDATION

Consider removing the if conditional around the `applyDeltaToVirtualInventoryForPositions` function so that if the migration should need to be reversed, it safely can be.

At a later time, once the migration is proven to be safe, the conditional can be added, or the old code removed entirely.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

oracleProviderUpdatedAt Ineffective Validation

CATEGORY	Validation
LOCATION	Config.sol
REVIEW	Remediation Review

— DESCRIPTION

The `setOracleProviderForFeeHandlerToken` function the `oracleProviderUpdatedAt` validation is keyed based on the token and provider combination. Therefore when a new provider is used to configure for a token the cooldown validation is not triggered.

The cooldown is only triggered when the same exact token and provider configuration is used again, which doesn't add as much protection as if the key was based purely off of the token. This however might limit practical configuration use-cases.

— RECOMMENDATION

Confirm that the current validation is intended and be aware that it is not as constricting as it could be.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Virtual Inventory Keys Are Allowed Base Keys

CATEGORY	Warning
LOCATION	Config.sol
REVIEW	Remediation Review

DESCRIPTION

In the `Config` contract the `VIRTUAL_INVENTORY_FOR_POSITIONS` and `VIRTUAL_INVENTORY_FOR_POSITIONS_IN_TOKENS` keys are declared as allowed base keys which can be configured by the config keeper.

However this may be a mechanism through which the config keeper is able to cause liquidations, by assigning a virtual inventory value that forces the maximum allowable magnitude negative price impact to affect the overall health of positions and force many on the exchange into liquidation.

RECOMMENDATION

Consider adding guardrails on the configuration values allowed for the virtual inventory related keys since these are system accounting variables. Otherwise consider removing them as an `allowedBaseKey`.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Dangerous OI In Tokens Base Key

CATEGORY	Warning
LOCATION	Config.sol
REVIEW	Remediation Review

— DESCRIPTION

The `USE_OPEN_INTEREST_IN_TOKENS_FOR_BALANCE` key is whitelisted in the `allowedBaseKey` mapping, however this may serve as a mechanism for the config keeper to liquidate positions or otherwise perturb the balancing mechanism of the exchange.

After the migration has taken place it is imperative that this value is not reset to false, as the previous open interest tracking may be inaccurate, especially in the case of the virtual inventory, which may lead to the incorrect liquidation of positions.

— RECOMMENDATION

Consider removing this key from the Config contract shortly after performing the migration.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Positive Price Impact Capped On Cost

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review

DESCRIPTION

In the `capPositiveImpactUsdByMaxPositionImpact` function the `maxPriceImpactUsdBasedOnMaxPriceImpactFactor` value is determined based upon the order's `sizeDeltaUsd`.

Therefore if an order, particularly a decrease order, has a small `sizeDeltaUsd` then the positive cap will be small. However this is not based upon the actual notional value that the order represents, which is determined by the `sizeInTokens * indexPrice`. After the migration is made, the `sizeInUsd` will also not be the metric which the price impact is based upon, creating a divergence between the capping logic and the way the price impact is computed.

Therefore in some edge cases where an account has made a lot of profit or a large loss, the capping may be inaccurate for the actual Notional size that is experiencing impact.

RECOMMENDATION

Consider refactoring the `capPositiveImpactUsdByMaxPositionImpact` cap validation to be based upon the notional size of the order instead of the cost delta of the order.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Negative Price Impact Capped On Cost

CATEGORY Warning
LOCATION Global
REVIEW Remediation Review

DESCRIPTION

In the `isPositionLiquidatable` function as well as the `DecreasePositionCollateralUtils` flow, negative price impact is capped based upon the order's `sizeDeltaUsd` rather than the actual notional value of the order.

After the migration this can cause a disconnect between the way price impact is calculated (now based on notional value) and the way it is capped (still based on cost).

RECOMMENDATION

Consider updating these negative caps in the `isPositionLiquidatable` function as well as the `DecreasePositionCollateralUtils` flow to be based upon the notional value of the order.

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Price Impact Pool Gaming Risk

CATEGORY	Gaming
LOCATION	Global
REVIEW	Remediation Review

— DESCRIPTION

Because the price impact calculations are now based upon the notional value of the difference in OIs, the value that is awarded or charged to a user for price impact is determined in some part by the index token price.

When the index token price goes downwards, the notional value of a `sizeInTokens` diff decreases and thus decreases the amount of price impact that is awarded or charged for creating that `sizeInTokens` diff. When the index token price goes upwards, the notional value of a `sizeInTokens` diff increases and thus increases the amount of price impact that is awarded or charged for creating that `sizeInTokens` diff.

There may exist some strategies to gain an overall positive net price impact through this, by causing larger imbalances in a market when token price trends down, and closing those imbalances when index token price trends up — and receiving more net positive impact for doing so than was paid in negative impact upon the creation of this imbalance.

— RECOMMENDATION

Be aware that impact pool gaming opportunities such as this may exist, though are not risk free. The capping of the lent amount can help to limit the impact of any such strategy if it were to be carried out.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

MaxCollateralSum Decrease DoS

CATEGORY	DoS
LOCATION	https://github.com/gmx-io/gmx-synthetics/blob/f04992d03cbfda878dae7c9148a15ef923206f5a/contracts/market/MarketUtils.sol#L1225-L1247
REVIEW	Remediation Review

DESCRIPTION

The system reverts now if the new `collateralSum` after applying some delta is bigger than the `maxCollateralSum`.

This check is also performed if the delta decreased the `collateralSum`. This will DoS the possibility to set the `maxCollateralSum` below the current `collateralSum` to prevent new increments and decrease it over time as in that case decreasing the collateral is DoSed too.

RECOMMENDATION

Be aware that impact pool gaming opportunities such as this may exist, though are not risk free. The capping of the lent amount can help to limit the impact of any such strategy if it were to be carried out.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Decrease Flow Always Uses Initial Margin Factor

CATEGORY	Warning
LOCATION	https://github.com/gmx-io/gmx-synthetics/blob/f04992d03cbfda878dae7c9148a15ef923206f5a/contracts/position/DecreasePositionUtils.sol#L334
REVIEW	Remediation Review

— DESCRIPTION

If a increase order does only increase the collateral of a position and not it's size and therefore decreases the overall leverage of the position. Than the collateral requirements are checked using the maintenance margin factor instead of the initial margin factor now.

However if a decrease order does only decrease the size of a position and not it's collateral and therefore decreases the overall leverage of the position.

Then the collateral requirements are still always checked with the initial margin factor. This is an asymmetry to be aware of.

— RECOMMENDATION

Consider to change or acknowledge this behavior.

However if the decision is made to change this be very careful as in the decrease case other condition are given as PnL is realized and fees apply. This may lead to not always improving the positions health and therefore such a change needs to be implemented with care.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.